

Propuesta-TFM-MarketplaceArtesanal

Autor: Ignacio Meléndez Uriz

Fecha: 23\06\2026

1. Qué es y qué hace la aplicación

Marketplace Artesanal es una plataforma multivendedor inspirada en Etsy a pequeña escala. Artesanos (cerámica, joyería, cuero, ilustración, textil, madera...) publican sus productos y los compradores los descubren, valoran y adquieren mediante un **checkout simulado** (sin pasarela de pago real para mantener el scope del proyecto lo más acotado y realista posible).

Tiene dos rasgos que la diferencian de una tienda online convencional:

1. **Un mismo usuario puede ejercer dos papeles.** Nace como comprador y puede darse de alta como vendedor abriendo su propia tienda. Es decir, los roles son **acumulables**.
2. **La confianza no la aporta una gran marca**, sino la *reputación* de cada pequeño vendedor, construida por software. Este es el *twist* único del proyecto (ver sección 8).

Funcionalidades dentro del alcance

- Catálogo de productos con categorías, filtros y búsqueda.
- Registro/login con JWT y gestión de roles acumulables.
- Alta de vendedor con su tienda y CRUD de productos/stock.
- Carrito y **checkout simulado** con ciclo de estados de pedido.
- Histórico de pedidos del comprador y gestión de pedidos del vendedor.
- Reseñas y valoraciones **verificadas** (solo reseña quien ha comprado).
- Sistema de **reputación** con score e insignias.
- Backoffice de administración (verificar vendedores, moderar, suspender).
- Wishlist / favoritos.

Fuera del alcance (decisión explícita)

- **Sin pagos reales.** El checkout es simulado: genera un pedido con estados PENDIENTE → PAGADO → ENVIADO → ENTREGADO . Se mantiene toda la complejidad interesante (roles, stock, reputación, pedidos) y solo se omite la pasarela de pago.

- Sin logística ni envíos reales (el coste de envío es un cálculo simulado).
- El panel de admin puede recortarse a lo mínimo (verificar vendedores) si el tiempo aprieta, concentrando el esfuerzo en el sistema de reputación.

2. Actores y roles

Un usuario nace como `BUYER` y puede añadir el rol `SELLER`. Los roles **suman** capacidades, no las restan: un vendedor o un admin también pueden comprar.

Capacidad	Comprador	Vendedor	Admin
Explorar catálogo y buscar	✓	✓	✓
Carrito y checkout	✓	✓	✓
Reseñar lo comprado	✓	✓	✓
Wishlist	✓	✓	✓
Abrir tienda / CRUD productos	—	✓	—
Panel de métricas de tienda	—	✓	—
Gestionar pedidos recibidos	—	✓	—
Verificar vendedores	—	—	✓
Gestionar categorías globales	—	—	✓
Moderar reseñas / suspender cuentas	—	—	✓

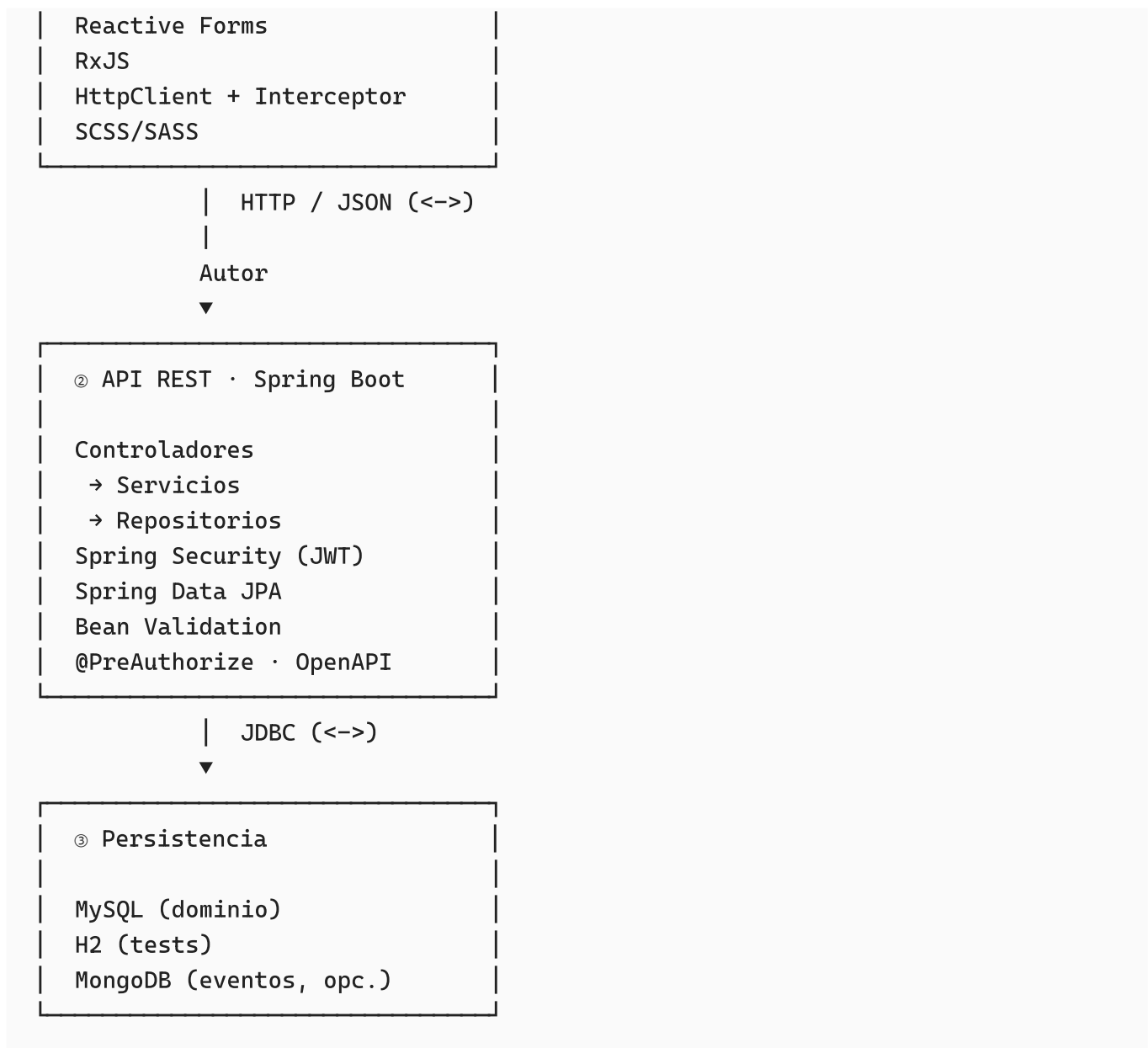
3. Arquitectura

Arquitectura clásica de **tres capas desacopladas**: una SPA en Angular consume una API REST en Spring Boot, que persiste en una base de datos relacional. Una colección NoSQL opcional almacena eventos para analítica (si es necesario o da tiempo a implementar).

① Cliente · Angular SPA

Components

Router + Guards



Stack tecnológico

Capa	Tecnologías
Frontend	Angular · Router + Guards · Reactive Forms · RxJS · HttpClient + Interceptor JWT · SCSS/SASS
Backend	Java 17 · Spring Boot · Spring Web · Spring Security (JWT) · Spring Data JPA · Bean Validation · @PreAuthorize · Swagger/OpenAPI
Persistencia	MySQL (dominio transaccional) · H2 (tests) · MongoDB (eventos, opcional)
Calidad y despliegue	JUnit · Mockito · Jasmine/Karma · Docker · Docker Compose · CI básico

Decisiones clave

Decisión	Alternativa descartada	Justificación
Monolito modular	Microservicios	Alcance realista para un TFM.
MySQL para el dominio	Solo NoSQL	El dominio es transaccional y relacional (pedidos, stock).
MongoDB solo para eventos	Todo en MySQL	Encaja con agregaciones para analítica/reputación.
JWT stateless	Sesiones de servidor	Encaja con SPA + REST; escala mejor y desacopla cliente/servidor.

4. Seguridad (front y back)

Tanto el front como el back incluyen seguridad. El backend es la autoridad; el frontend solo mejora la experiencia (oculta lo que el usuario no puede usar), nunca es la barrera real.

En el backend

- `SecurityFilterChain` con un **filtro JWT propio** que valida el token y popula el `SecurityContext`.
- Autorización a nivel de método con `@PreAuthorize("hasRole('SELLER')")`.
- **Comprobación de propiedad en el servicio**: no basta con tener el rol `SELLER` para editar un producto; debe ser **su** producto. Esto es seguridad a nivel de instancia, no solo de rol.
- **CORS** configurado para la SPA y **manejo global de errores** 401 / 403.

```
@PreAuthorize("hasRole('SELLER')")
public ProductDTO updateProduct(Long productId, ProductUpdatedDTO dto, User
current) {
    Product product = productRepository.findById(productId)
        .orElseThrow(() -> new NotFoundException("Producto no
    encontrado"));

    // Regla de propiedad: el producto debe ser de la tienda del usuario
    autenticado
    if (!product.getShop().getOwnerId().equals(current.getId())) {
        throw new AccessDeniedException("No puedes editar productos de otra
    tienda");
    }
}
```

```
}  
  // ... aplicar cambios  
}
```

En el frontend

- **Guards** (`CanActivate`) que protegen rutas por rol.
- **Interceptor** que adjunta el JWT y redirige al login ante un `401` .
- **Menús y botones** que se muestran/ocultan según el rol del usuario.
- **Lazy loading** del módulo de vendedor y del backoffice de administración.

Matriz de endpoints (resumen)

Endpoint	Método	Acceso	Comprobación extra
<code>/api/products</code>	GET	PÚBLICO	—
<code>/api/auth/register</code> · <code>/api/auth/login</code>	POST	PÚBLICO	—
<code>/api/cart</code> · <code>/api/checkout</code>	POST	BUYER	—
<code>/api/me/orders</code>	GET	BUYER	Solo sus pedidos
<code>/api/products/{id}/reviews</code>	POST	BUYER	Solo si lo compró
<code>/api/seller/products</code>	POST	SELLER	—
<code>/api/seller/products/{id}</code>	PUT · DELETE	SELLER	Producto de su tienda
<code>/api/admin/shops/{id}/verify</code>	POST	ADMIN	—
<code>/api/admin/users/{id}/suspend</code>	POST	ADMIN	—

Estrategia: RBAC + ownership

Se elige **RBAC (Role-Based Access Control)** como base (`BUYER` / `SELLER` / `ADMIN`) y se complementa con **comprobaciones de propiedad** a nivel de instancia. Esta combinación cubre el

"¿qué rol tienes?" y el "¿eres el dueño?" sin la complejidad de un motor ABAC completo.

5. Docker

Se usa **Docker y Docker Compose**. El proyecto se orquesta como un monorepo con servicios contenedorizados, lo que garantiza un **despliegue reproducible**.

- `docker-compose.yml` levanta los servicios `api` (Spring Boot) + `mysql` (y `mongo` opcional).
 - El backend incluye su propio `Dockerfile`.
 - Un perfil de configuración específico (`application-docker.yml`) apunta a la base de datos dentro de la red de contenedores.
-

6. Base de datos

Se usa **persistencia de datos** utilizando:

- **MySQL** — base de datos principal para el **dominio transaccional**: usuarios, tiendas, productos, categorías, pedidos y reseñas. Es relacional porque el dominio lo es (integridad referencial, transacciones, stock).
- **H2** en memoria — solo para los **tests** (arranque rápido, sin dependencias externas).
- **MongoDB** (*opcional y si da tiempo*) — para **eventos** (clics, búsquedas) que alimentan analítica y el sistema de reputación mediante el Aggregation Pipeline.

El acceso a datos se hace con **Spring Data JPA** (repositorios) sobre MySQL.

7. Opciones de búsqueda

La búsqueda y el filtrado del catálogo son una funcionalidad central. Opciones previstas:

- **Búsqueda por texto** sobre título (y descripción) del producto.
- **Filtro por categoría** (pills/categorías que filtran la rejilla en vivo).
- **Filtro por rango de precio**.
- **Filtro por valoración** (rating mínimo).
- **Filtro por tienda verificada** / reputación del vendedor.
- **Ordenación** por precio, valoración o novedad.
- **Paginación** de resultados (rendimiento con catálogos grandes).

A nivel técnico se resolverá con **Spring Data JPA** (derived queries y/o `@Query`), y queda abierta la posibilidad de usar **Specifications/Criteria** para combinar filtros dinámicos. La parte de rendimiento (índices, paginación) conecta con el temario de búsqueda y optimización.

8. El *twist*: sistema de reputación y confianza

El punto diferenciador frente a un e-commerce normal es el **Sistema de Reputación y Confianza**. Trata de responder a la pregunta:

¿cómo genera confianza una plataforma entre dos desconocidos?

Para ello, se plantea el diseño de un **score de reputación (0–100)** por tienda que combina varias métricas:

```
score = 0.40 · ratingMedioNorm    // calidad percibida (reseñas)
      + 0.30 · volumenVentasNorm  // trayectoria (ventas completadas)
      + 0.20 · antiguedadNorm     // veteranía en la plataforma
      - 0.10 · tasaIncidencias    // penalización por disputas
```

Una posibilidad a añadir si el tiempo lo permite es acompañarlo de:

- **Insignias** (gamificación): Vendedor verificado, Artesano destacado (score > 85), Respuesta rápida (< 24 h), +100 ventas.
- **Prevención de fraude**: solo reseña quien ha comprado, una reseña por comprador y producto, detección de patrones sospechosos, moderación del admin.
- **Análisis comparativo** con plataformas reales (eBay, Wallapop, Etsy, Stack Overflow) sobre qué señales usan y cómo previenen el fraude.

El cálculo puede hacerse *on-read* o mediante tareas programadas (@Scheduled), y los eventos que lo alimentan son un caso ideal para el Aggregation Pipeline de MongoDB.

9. Esquema de la base de datos (a grosso modo)

Entidades principales

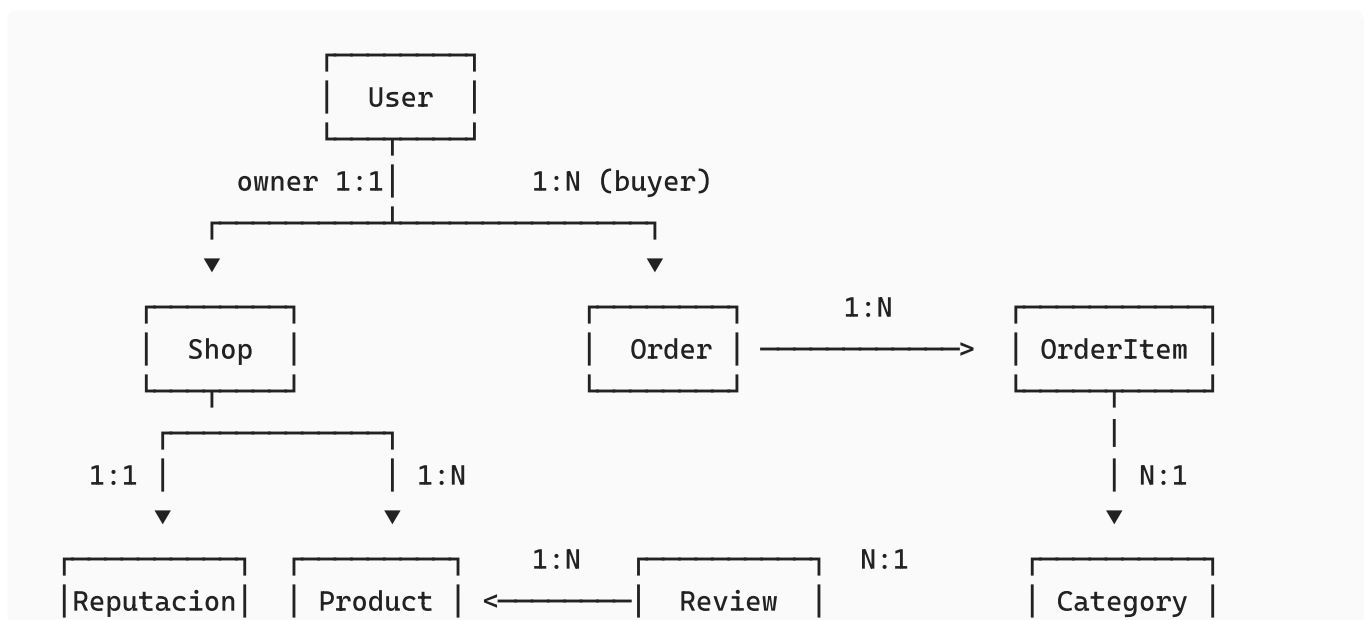
Entidad	Campos clave
User	id (PK), email (único), passwordHash , nombre , roles[] (BUYER / SELLER / ADMIN)
Shop	id (PK), ownerId (FK→User), nombre , descripcion , verificada (bool)
Product	id (PK), shopId (FK→Shop), categoriaId (FK→Category), titulo , precio , stock , imagenes[]
Category	id (PK), nombre , slug

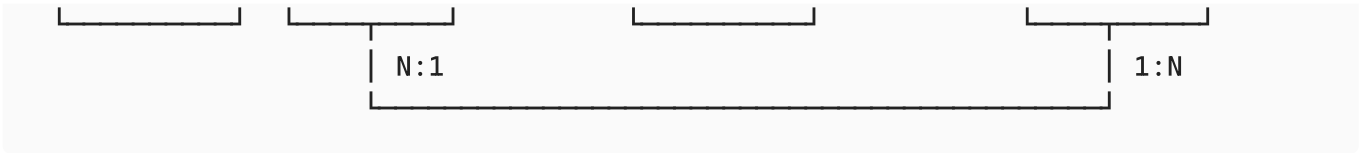
Entidad	Campos clave
Order	id (PK), buyerId (FK→User), estado (enum), total , fecha
OrderItem	id (PK), orderId (FK→Order), productId (FK→Product), precio (congelado), cantidad
Review	id (PK), buyerId (FK→User), productId (FK→Product), rating (1–5), comentario , fecha
Reputacion ★	shopId (FK→Shop), score (0–100), nVentas , ratingMedio , insignias[]

Relaciones

Relación	Cardinalidad
User — Shop	1:1 (un usuario tiene como mucho una tienda)
Shop — Product	1:N
Category — Product	1:N
User (buyer) — Order	1:N
Order — OrderItem	1:N
Product — OrderItem	1:N
User — Review	1:N
Product — Review	1:N
Shop — Reputacion	1:1 (agregado calculado)

Diagrama entidad-relación (resumen)





Nota: el `precio` se "**congela**" en `OrderItem` en el momento de la compra para que cambios posteriores en `Product.precio` no alteren pedidos históricos.